

LẬP TRÌNH PYTHON CHO MACHINE LEARNING

Từ Python cơ bản đến Machine Learning

Ngô Hoàng Dũng - Lê Hoàng Ân

Ngày 7 tháng 9 năm 2026

Mục lục

1	Giới thiệu về Python	1
1.1	Python là gì?	1
1.2	Lịch sử phát triển của Python	1
1.2.1	Python 1.x	1
1.2.2	Python 2.x	2
1.2.3	Python 3.x	2
1.3	Guido van Rossum – tác giả của Python	2
1.4	Tên gọi Python	2
1.5	Python ngày nay	2
1.6	Tóm tắt chương	3
2	Cú pháp Python cơ bản	4
2.1	Biến và Kiểu dữ liệu	4
2.1.1	Các kiểu dữ liệu cơ bản	4
2.2	Cấu trúc dữ liệu nâng cao	4
2.2.1	List (Danh sách)	5
2.3	Cấu trúc điều khiển và Vòng lặp	5
2.3.1	Câu lệnh điều kiện if-else	5
2.4	Tóm tắt chương	5
3	Lập trình hướng đối tượng trong Python	6
3.1	Lớp và Đối tượng (Class and Object)	6
3.2	Các phương thức đặc biệt (Magic Methods)	6
3.2.1	Phương thức <code>__init__</code> và <code>__call__</code>	6
3.3	Tính Kế thừa (Inheritance)	7
3.4	Quản lý thuộc tính: Siêu tham số (Hyperparameters) và Tham số học được (Learned Parameters)	8
3.5	Thiết kế Custom Estimator chuẩn Scikit-Learn	8
3.6	Mẫu thiết kế Pipeline	9
3.7	Tóm tắt chương	9
4	Hồi quy tuyến tính (Linear Regression)	10
4.1	Mô hình Hồi quy tuyến tính đơn biến	10
4.2	Hàm mất mát (Loss Function)	10
4.3	Cài đặt Hồi quy tuyến tính với Scikit-Learn	10
4.4	Tóm tắt chương	11

Chương 1: Giới thiệu về Python

1.1 Python là gì?

Python là một ngôn ngữ lập trình bậc cao, thông dịch và đa mục đích. Python được thiết kế với mục tiêu tạo ra một ngôn ngữ có cú pháp rõ ràng, dễ đọc và dễ học.

Python hỗ trợ nhiều mô hình lập trình, trong đó có lập trình thủ tục, lập trình hướng đối tượng và lập trình hàm. Nhờ cú pháp đơn giản cùng hệ sinh thái thư viện phong phú, Python được sử dụng rộng rãi trong nhiều lĩnh vực như phát triển phần mềm, phát triển Web, khoa học dữ liệu, trí tuệ nhân tạo, Machine Learning và tự động hóa.

Một chương trình Python đơn giản:

```
1 print("Hello , Python!")
```

Cú pháp của Python có xu hướng sử dụng khoảng trắng để thể hiện cấu trúc của chương trình. Ví dụ:

```
1 if age >= 18:
2     print("Adult")
3 else:
4     print("Under 18")
```

Cách tổ chức này giúp mã nguồn Python có cấu trúc rõ ràng và dễ đọc.

1.2 Lịch sử phát triển của Python

Python được tạo ra bởi Guido van Rossum vào cuối những năm 1980 và được phát triển trong những năm đầu của thập niên 1990.

Guido van Rossum bắt đầu phát triển Python như một dự án kế thừa từ ngôn ngữ lập trình ABC. Mục tiêu của ông là xây dựng một ngôn ngữ có cú pháp đơn giản, dễ sử dụng nhưng vẫn đủ mạnh để phát triển các chương trình thực tế.

Phiên bản Python đầu tiên được công bố rộng rãi là Python 0.9.0, phát hành vào năm 1991. Phiên bản này đã có nhiều đặc điểm quan trọng của Python hiện đại, bao gồm các kiểu dữ liệu như danh sách (list), chuỗi (string), từ điển (dictionary), hàm và cơ chế xử lý ngoại lệ.

1.2.1 Python 1.x

Python 1.0 được phát hành vào năm 1994. Trong giai đoạn này, ngôn ngữ tiếp tục được phát triển và bổ sung nhiều tính năng quan trọng.

1.2.2 Python 2.x

Python 2.0 được phát hành vào năm 2000. Đây là một cột mốc quan trọng trong lịch sử Python, với nhiều cải tiến về ngôn ngữ và khả năng xử lý dữ liệu.

Python 2 tiếp tục được sử dụng trong nhiều năm. Tuy nhiên, Python 2 cuối cùng đã được ngừng hỗ trợ chính thức vào năm 2020.

1.2.3 Python 3.x

Python 3.0 được phát hành vào năm 2008. Đây là một phiên bản được thiết kế để cải thiện ngôn ngữ và khắc phục một số hạn chế của Python 2.

Python 3 không hoàn toàn tương thích ngược với Python 2. Vì vậy, quá trình chuyển đổi từ Python 2 sang Python 3 diễn ra trong nhiều năm.

Ngày nay, Python 3 là dòng phiên bản chính được sử dụng và phát triển.

1.3 Guido van Rossum – tác giả của Python

Guido van Rossum là lập trình viên người Hà Lan và là người tạo ra ngôn ngữ lập trình Python.

Ông bắt đầu phát triển Python vào cuối năm 1989 tại Centrum Wiskunde & Informatica (CWI) ở Hà Lan. Phiên bản đầu tiên của Python được công bố vào năm 1991.

Guido van Rossum giữ vai trò chính trong việc định hướng sự phát triển của Python trong nhiều năm. Ông từng được cộng đồng Python gọi bằng danh hiệu “Benevolent Dictator For Life”, thường được viết tắt là BDFL.

Năm 2018, Guido van Rossum tuyên bố từ bỏ vai trò BDFL. Sau đó, quá trình định hướng phát triển Python được chuyển sang mô hình quản trị dựa trên Python Steering Council.

1.4 Tên gọi Python

Tên “Python” không xuất phát từ loài rắn như nhiều người thường nghĩ. Guido van Rossum đặt tên ngôn ngữ theo chương trình hài kịch truyền hình của Anh có tên *Monty Python’s Flying Circus*.

Tên gọi ngắn gọn “Python” được lựa chọn vì Guido van Rossum là người yêu thích chương trình này.

1.5 Python ngày nay

Python hiện được sử dụng trong rất nhiều lĩnh vực của công nghệ thông tin. Một số lĩnh vực tiêu biểu bao gồm:

- Phát triển phần mềm.
- Phát triển Web.
- Khoa học dữ liệu.
- Trí tuệ nhân tạo.

- Machine Learning.
- Deep Learning.
- Tự động hóa.
- Tính toán khoa học.

Đặc biệt, hệ sinh thái Python có nhiều thư viện mạnh như NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow và PyTorch. Đây là một trong những yếu tố quan trọng giúp Python trở thành ngôn ngữ phổ biến trong khoa học dữ liệu và Machine Learning.

1.6 Tóm tắt chương

Trong chương này, chúng ta đã tìm hiểu:

- Khái niệm cơ bản về Python.
- Lịch sử hình thành và phát triển của Python.
- Các giai đoạn Python 1.x, Python 2.x và Python 3.x.
- Guido van Rossum – người tạo ra Python.
- Nguồn gốc tên gọi Python.
- Một số lĩnh vực sử dụng Python hiện nay.

Chương 2: Cú pháp Python cơ bản

Chương này trình bày các khái niệm nền tảng trong lập trình Python, bao gồm biến, kiểu dữ liệu, cấu trúc điều khiển và hàm. Đây là các kiến thức bắt buộc trước khi đi vào xử lý dữ liệu và Machine Learning.

2.1 Biến và Kiểu dữ liệu

Trong Python, biến được khởi tạo tự động khi gán giá trị. Python là ngôn ngữ dynamically typed (định kiểu động).

Định nghĩa 2.1 (Biến trong Python). *Biến là tên gọi đại diện cho một vùng nhớ chứa dữ liệu. Giá trị của biến có thể thay đổi trong quá trình thực thi chương trình.*

2.1.1 Các kiểu dữ liệu cơ bản

Python hỗ trợ nhiều kiểu dữ liệu mặc định:

- **Integer** (int): Số nguyên, ví dụ: `x = 10`.
- **Float** (float): Số thực, ví dụ: `pi = 3.14159`.
- **String** (str): Chuỗi ký tự, ví dụ: `name = "Machine Learning"`.
- **Boolean** (bool): Giá trị đúng/sai (True hoặc False).

Ví dụ 2.1. *Ví dụ về khai báo và kiểm tra kiểu dữ liệu trong Python:*

Listing 2.1: Khai báo biến và hiển thị kiểu dữ liệu

```
1 a = 100
2 b = 3.14
3 c = "Python ML"
4 is_ready = True
5
6 print(type(a))    # Output: <class 'int'>
7 print(type(b))    # Output: <class 'float'>
```

2.2 Cấu trúc dữ liệu nâng cao

Đối với Machine Learning, 3 cấu trúc dữ liệu cơ bản hay dùng nhất bao gồm List, Tuple và Dictionary.

2.2.1 List (Danh sách)

List là tập hợp các phần tử có thứ tự và có thể thay đổi (mutable).

Listing 2.2: Thao tác cơ bản trên List

```
1 # Khởi tạo list
2 features = [1.2, 3.4, 5.1, 2.8]
3
4 # Truy cập và thay đổi phần tử
5 features[0] = 0.9
6 features.append(4.5)
7
8 print("List sau khi cập nhật:", features)
```

2.3 Cấu trúc điều khiển và Vòng lặp

2.3.1 Câu lệnh điều kiện if-else

Listing 2.3: Sử dụng cấu trúc điều kiện

```
1 score = 0.85
2
3 if score >= 0.8:
4     print("Mô hình đạt độ chính xác cao")
5 else:
6     print("Mô hình cần thiết hoặc tối ưu thêm")
```

2.4 Tóm tắt chương

Trong chương này, chúng ta đã nắm được:

- Cách khai báo biến và các kiểu dữ liệu cơ bản.
- Cấu trúc dữ liệu List, Tuple, Dictionary.
- Vòng lặp và cấu trúc rẽ nhánh trong Python.

Chương 3: Lập trình hướng đối tượng trong Python

Lập trình hướng đối tượng (Object-Oriented Programming - OOP) là phương pháp luận lập trình đóng vai trò then chốt trong việc thiết kế các thư viện Machine Learning hiện đại như Scikit-Learn, PyTorch và TensorFlow. Chương này trình bày các khái niệm OOP cốt lõi và hướng dẫn cách ứng dụng OOP để tự đóng gói một mô hình Machine Learning.

3.1 Lớp và Đối tượng (Class and Object)

Định nghĩa 3.1 (Lớp và Đối tượng). *Lớp (Class)* là khuôn mẫu định nghĩa các thuộc tính (dữ liệu) và phương thức (hành vi). *Đối tượng (Object)* là một thể hiện cụ thể được khởi tạo từ lớp đó.

Ví dụ 3.1. Khai báo một lớp cơ bản lưu trữ mẫu dữ liệu:

Listing 3.1: Khai báo Class và tạo Object trong Python

```
1 class DataSample:
2     def __init__(self, feature_name, value):
3         self.feature_name = feature_name # Thuộc tính
4         self.value = value
5
6     def display_info(self):                # Phương thức
7         print(f"Feature: {self.feature_name}, Value: {self.value}")
8
9 # Khởi tạo đối tượng
10 sample = DataSample(feature_name="Age", value=25)
11 sample.display_info()
```

3.2 Các phương thức đặc biệt (Magic Methods)

Các phương thức có cú pháp hai dấu gạch dưới (`__method__`) giúp tùy biến hành vi của đối tượng trong Python.

3.2.1 Phương thức `__init__` và `__call__`

- `__init__`: Hàm khởi tạo (constructor), tự động thực thi khi tạo mới một đối tượng.

- `__call__`: Cho phép gọi trực tiếp một đối tượng giống như một hàm, thường dùng khi thiết kế các Layer trong Neural Network.

Listing 3.2: Sử dụng phương thức `__call__`

```

1 class Multiplier:
2     def __init__(self, factor):
3         self.factor = factor
4
5     def __call__(self, x):
6         return x * self.factor
7
8 # Su dụng như một hàm
9 double = Multiplier(factor=2)
10 print(double(5)) # Output: 10

```

3.3 Tính Kế thừa (Inheritance)

Tính kế thừa cho phép tạo một lớp mới dựa trên lớp đã có, giúp tái sử dụng mã nguồn và chuẩn hóa giao diện của các bộ biến đổi dữ liệu (Transformers).

Listing 3.3: Kế thừa lớp BaseTransformer trong tiền xử lý dữ liệu

```

1 class BaseTransformer:
2     def fit(self, arr):
3         raise NotImplementedError("Phương thức fit() bắt buộc phải
4             được triển khai ở lớp con.")
5
6     def transform(self, arr):
7         raise NotImplementedError("Phương thức transform() bắt buộc
8             phải triển khai ở lớp con.")
9
10    def fit_transform(self, arr):
11        self.fit(arr)
12        return self.transform(arr)
13
14 class MinMaxScaler(BaseTransformer):
15     def fit(self, arr):
16         self.min_ = min(arr)
17         self.max_ = max(arr)
18         return self
19
20     def transform(self, arr):
21         return [(x - self.min_) / (self.max_ - self.min_) for x in
22             arr]
23
24     def inverse_transform(self, arr):
25         return [x * (self.max_ - self.min_) + self.min_ for x in arr]

```

3.4 Quản lý thuộc tính: Siêu tham số (Hyperparameters) và Tham số học được (Learned Parameters)

Trong thiết kế Machine Learning, ta cần phân biệt rõ giữa siêu tham số (cấu hình trước khi huấn luyện) và tham số học được (được sinh ra sau khi gọi hàm fit).

Listing 3.4: Bắt lỗi an toàn khi quản lý tham số học được

```

1 class MinMaxScaler(BaseTransformer):
2     def __init__(self, feature_range=(0, 1)):
3         self.feature_range = feature_range # Hyperparameter
4
5     def fit(self, arr):
6         self.min = float(min(arr)) # Learned Parameter
7         self.max = float(max(arr)) # Learned Parameter
8         return self
9
10    def transform(self, arr):
11        # Kiểm tra an toàn: Đảm bảo mô hình đã được fit()
12        if not hasattr(self, 'min') or not hasattr(self, 'max'):
13            raise RuntimeError("Lỗi: Bạn phải gọi hàm fit() trước khi
14                               gọi transform()!")
15
16        low, high = self.feature_range
17        std = [(x - self.min) / (self.max - self.min) for x in arr]
18        return [x * (high - low) + low for x in std]

```

3.5 Thiết kế Custom Estimator chuẩn Scikit-Learn

Trọng tâm ứng dụng OOP trong Machine Learning là đóng gói thuật toán thành một Class có các giao diện chuẩn (fit, predict, evaluate). Dưới đây là triển khai mô hình Hồi quy tuyến tính đơn biến (Simple Linear Regression) bằng công thức đại số.

Listing 3.5: Đóng gói thuật toán Simple Linear Regression

```

1 class SimpleLinearRegression:
2     def __init__(self):
3         self.w0 = 0
4         self.w1 = 0
5
6     def fit(self, X, y):
7         n = len(X)
8         mean_x = sum(X) / n
9         mean_y = sum(y) / n
10
11        numerator = sum((X[i] - mean_x) * (y[i] - mean_y) for i in
12                        range(n))
13        denominator = sum((X[i] - mean_x) ** 2 for i in range(n))
14
15        if denominator == 0:

```

```

15         raise ValueError("Bao loi: Phuong sai cua X bang 0, khong
16             the tinh toan.")
17
18     self.w1 = numerator / denominator
19     self.w0 = mean_y - (self.w1 * mean_x)
20     return self
21
22     def predict(self, X):
23         return [(self.w1 * x_i + self.w0) for x_i in X]
24
25     def evaluate(self, X, y):
26         predictions = self.predict(X)
27         mse = sum((predictions[i] - y[i]) ** 2 for i in range(len(y)))
28             ) / len(y)
29         return mse

```

3.6 Mẫu thiết kế Pipeline

Pipeline là lớp quản lý chuỗi các bước biến đổi dữ liệu nối tiếp nhau, cho phép tự động hóa toàn bộ quy trình tiền xử lý trước khi đưa vào mô hình dự đoán.

Listing 3.6: Triển khai cấu trúc Pipeline cơ bản

```

1 class Pipeline:
2     def __init__(self, steps):
3         self.steps = steps
4
5     def fit_transform(self, arr):
6         data = arr
7         for transformer in self.steps:
8             data = transformer.fit_transform(data)
9         return data
10
11     def transform(self, arr):
12         data = arr
13         for transformer in self.steps:
14             data = transformer.transform(data)
15         return data

```

3.7 Tóm tắt chương

- Khái niệm Lớp, Đối tượng và tư duy lập trình hướng đối tượng.
- Ứng dụng các Magic Methods (`__init__`, `__call__`) trong xây dựng Framework.
- Nguyên lý Kế thừa giúp tái sử dụng và quản lý mã nguồn linh hoạt.
- Đóng gói thuật toán Machine Learning thành Class theo giao diện tiêu chuẩn.
- Phân biệt rõ ràng siêu tham số và tham số học được thông qua các thuộc tính của đối tượng

Chương 4: Hồi quy tuyến tính (Linear Regression)

Hồi quy tuyến tính là một trong những thuật toán học có giám sát (Supervised Learning) cơ bản và phổ biến nhất trong Machine Learning, dùng để dự đoán giá trị đầu ra liên tục.

4.1 Mô hình Hồi quy tuyến tính đơn biến

Định nghĩa 4.1 (Hồi quy tuyến tính đơn biến). Cho tập dữ liệu $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. Mô hình biểu diễn mối quan hệ giữa biến độc lập x và biến phụ thuộc y theo dạng tuyến tính:

$$y = w_0 + w_1x + \epsilon \quad (4.1)$$

trong đó w_0 là hệ số chặn (intercept), w_1 là hệ số góc (slope), và ϵ là sai số ngẫu nhiên.

4.2 Hàm mất mát (Loss Function)

Để tìm các hệ số w_0, w_1 tối ưu, người ta sử dụng sai số bình phương trung bình (Mean Squared Error - MSE):

$$J(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (y_i - (w_0 + w_1x_i))^2 \quad (4.2)$$

Mục tiêu của bài toán là tìm (w_0, w_1) sao cho $J(w_0, w_1)$ đạt giá trị nhỏ nhất.

4.3 Cài đặt Hồi quy tuyến tính với Scikit-Learn

Dưới đây là đoạn code Python minh họa việc xây dựng mô hình Linear Regression trên tập dữ liệu thực tế:

Listing 4.1: Thực thi Linear Regression bằng Scikit-Learn

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3
4 # 1. Tạo dữ liệu giả định
5 X = np.array([[1], [2], [3], [4], [5]])
6 y = np.array([2.1, 3.9, 6.1, 8.2, 9.9])
7
```

```
8 # 2. Khởi tạo và huấn luyện mô hình
9 model = LinearRegression()
10 model.fit(X, y)
11
12 # 3. In kết quả trong số  $w_0$  và  $w_1$ 
13 print("Hệ số  $w_1$  (slope):", model.coef_[0])
14 print("Hệ số  $w_0$  (intercept):", model.intercept_)
15
16 # 4. Dự đoán giá trị mới
17 y_pred = model.predict([[6]])
18 print("Dự đoán cho X=6:", y_pred[0])
```

4.4 Tóm tắt chương

- Bản chất mô hình Hồi quy tuyến tính và bài toán tối ưu hàm MSE.
- Phương pháp hạ độ dốc (Gradient Descent) để cập nhật trọng số.
- Thao tác huấn luyện mô hình bằng thư viện `scikit-learn`.